I could not find a way to use my proteomics dataset for a generalized linear model as the data described normalized abundances of proteins in the phloem sap of plants following three treatment conditions. Since this is not a count, proportion, or binary outcome, and is a transformed continuous response I do not believe it can be fit to a generalized linear model. In searching for ways to use this data in a generalized linear model, I found that proteomics data analysis seems to exclusively use linear models for analysis.

Because of this, I opted to use bacterial-level data, which is a kind of count data for the level of live bacteria extracted from infected plant tissue. This data is of interest in determining if treatment with a small peptide, flg22, which is recognized by plant immune receptors, is sufficient to induce Systemic acquired resistance (A form of systemic defense priming) in *Arabidopsis.* The hypothesis for the experiment was that induction of PAMP-triggered immunity signaling pathways through flg22 infiltration would be sufficient to induce SAR long-distance signaling and prime distant leaves for resistance. After collecting data, we observed flg22 treatment appeared to have a weaker fold change than biologically-induced SAR using the avirulent pathogen *Pseudomonas syringae* pv. *tomato*. Therefore, I would like to use a generalized linear model to test the hypothesis that flg22 treatment induces a weaker reduction in bacterial growth in distant leaves than biologically induced SAR.

Fitting the Generalized Linear Model

I began by compiling data from three independent experiments to increase the number of observations for each treatment. This resulted in each treatment comprising 9 observations rather than the 3 observations collected per treatment per experiment. I first fit the model using exclusively treatment as a predictor variable, using a quasipoisson family function, as it is count data. I also fit a model with experiment number as a parameter as I suspected there would be variation between experiments which could help to explain noise in the model. I tested the models against another using anova, and found the model with the experiment parameter did a better job at explaining the data.

```
BQ_glm <- glm(population ~ treatment,
              data = BQ_compile,
              family = quasipoisson(link = "log"))

BQ_glm2 <- update(BQ_glm, . ~ treatment + experiment)

anova(BQ_glm, BQ_glm2, test = “f”)
```

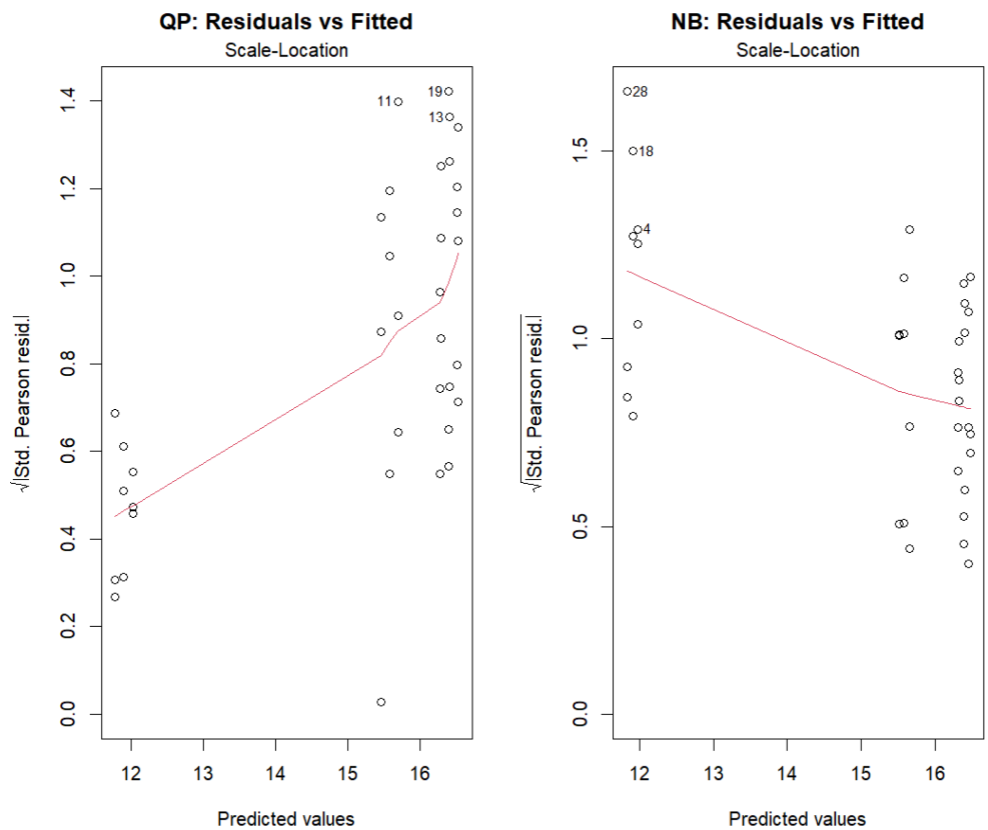
I also created a model using the negative binomial family function since Dr. Bolker said it is usually the better choice if you can fit it, and it also works for count data. Frankly, I am uncertain why to choose one over the other.

```
BQ_glm_nb <- glm.nb(population ~ treatment + experiment,
                    data = BQ_compile)
```

I then checked for overdispersion by calculating (residual deviance)/(residual df) for both models. The quasipoisson model was 263272.6, which is probably concerning if it should be ~ 1. For the negative binomial model this value was 1.17, which is perhaps worrisome, but much better than the quasipoisson model.

Diagnostic Plots

```
par(mfrow=c(1,2))
plot(BQ_glm2, which=3, main="QP: Residuals vs Fitted")
plot(BQ_glm_nb2, which=3, main="NB: Residuals vs Fitted")
```

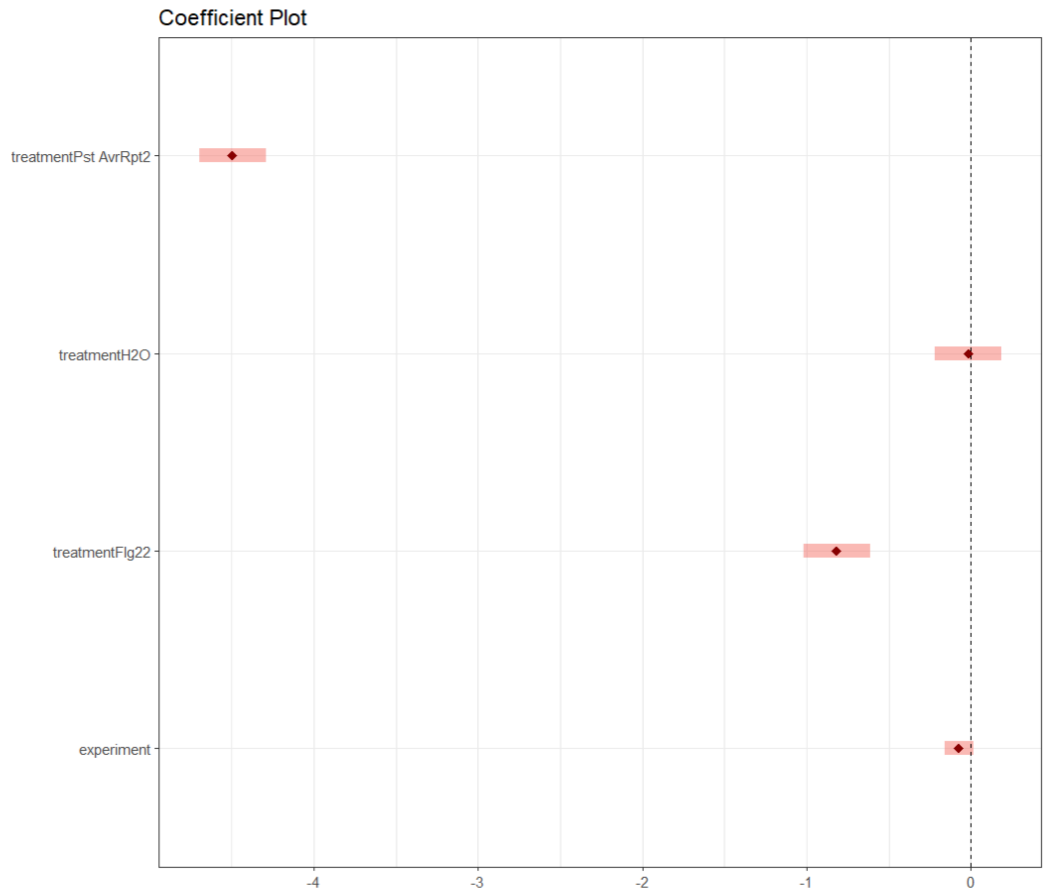


The scale-location plots for the quasi-Poisson (left) and negative binomial (right) models show how residual variances change with fitted values. These plots are used to assess the assumption of homoscedasticity in linear models, but for generalized linear models, I believe they demonstrate the relationship between predicted mean values and variance. Ideally, the plots should show a relatively flat line.

The quasi-Poisson plot shows a moderate to strong upward trend, indicating that residual variance is increasing as predicted values increase. I believe that this suggests that the quasi-Poisson model is underestimating the variance at high predicted values. In contrast, the negative binomial model is flatter, but has a weak downward trend. This indicates that the negative binomial model may be slightly overestimating variance at large predicted values. I believe this means the negative binomial model provides a better fit than the quasi-Poisson model.

Inferential Plot

```
dotwhisker::dwplot(BQ_glm_nb2, …
```



This dotwhisker plot from the dotwhisker package plots the coefficients of the negative binomial linear model. The treatmentPst AvrRpt2 coefficient is large and negative, indicating that treatment with a SAR-inducing pathogen like *Pst avrRpt2* has a strong negative effect on the response variable (bacterial population) relative to the base condition (treatmentMgCl2 i.e., mock). As expected, the treatmentH2O coefficient straddles 0 on the plot, suggesting it does not have a clear effect on bacterial level in distant leaves. The coefficient for treatmentFlg22 is small and negative, suggesting it has a weaker effect on bacterial level than *Pst avrRpt2*, and the effect is clearly not zero. Finally the experiment coefficient appears slightly negative, but with confidence intervals that extend past zero, suggesting the effect is not clear. All three of the experiments compiled to collect this data were done within a span of approximately 3 weeks in the summer, when experimental variability is lowest. As such, a small, unclear effect from the experiment coefficient, suggesting low experimental variability, is not surprising.

Looking at a plot of emmeans using 83.4% non-overlap intervals, I can see that there is a small, but clear difference between the predicted values for mock treatments ($MgCl_2$ or H_2O) and Flg22. Additionally, the effect of Flg22 on bacterial level is clearly less than biologically-induced SAR. As expected, there is no clear difference in the predicted values for the mock-treated conditions.

```
emms <- emmeans(BQ_glm_nb2, ~ treatment, level = 0.834)
plot_data <- as.data.frame(emms)
ggplot(plot_data, aes(x = treatment, y = emmean)) +
  geom_point(size = 2.5,
             position = position_dodge(width = 0.25)) +
  geom_line(linewidth = 1,
            position = position_dodge(width = 0.25),
            alpha = 0.75)+
  geom_errorbar(aes(x = treatment, ymin = asymp.LCL,
                                 ymax = asymp.UCL),
...
```